The Coding Agent Era

OpenCode · **OpenClaw** · GitHub Copilot

Token Economics Observations and Practical Developer Guide Across Three Major Platforms

Kinfey Lo · Microsoft Senior Cloud Advocate

May 2026

AGENDA

What we'll cover today

01 **Coding Agent Introduction**

Why the 2026 dev workflow is being reshaped

02 **Side-by-side breakdown of 3 platforms**

OpenCode · OpenClaw · GitHub Copilot

03 **Token Consumption Profile**

Billing models, multipliers, optimization headroom

04 **Selection & Cost Optimization**

Scenario fit + practical recommendations

Chapter 1

What is a Coding Agent?

No longer "completion" — but an "agent".

A Coding Agent can plan, invoke tools, read files, and run shell — driving tasks forward through multi-step loops until the goal is reached or a human intervenes.

Goal-driven

Understand high-level intent, autonomously decompose tasks

Tool calling

Read/write files, run commands, search code

Multi-turn loop

Plan → Act → Observe → Reflect

Landscape

Three platforms · Three philosophies

OpenCode

Open · Terminal-first

- Built by sst, fully open source
- Model-agnostic (BYOM)
- Build / Plan dual Agent

Cost = direct model passthrough

OpenClaw / Pi

Minimal · Self-evolving

- Only 4 built-in tools
- Pi extends by self-rewriting
- WhatsApp end-to-end experience

Cost = minimal kernel

GitHub Copilot

Enterprise · Integrated

- IDE Agent + Cloud Agent
- Premium Request billing
- Transitioning to Token quotas

Cost = plan + quota

Chapter 2 / OpenCode

A terminal-native open-source Coding Agent

*Open-sourced by the sst team — **fully open · model-agnostic.***

OpenCode treats the Coding Agent as a standalone CLI tool, embedded directly into the developer's existing terminal workflow. It's not tied to any specific model — switch freely between Anthropic, OpenAI, local Ollama, and more.

License	**MIT, fully open source**	Interface	**TUI / Terminal**
Model support	**Multiple APIs + local**	Extension model	**Custom Agents / tools**

OpenCode / Architecture

Build · Plan · Sub-agent collaboration

Build

PRIMARY · All tools

Default development primary agent — full read, write, execute, and search permissions.

Plan

PRIMARY · Restricted

Read-only planning agent — for review, design, and avoiding mistaken operations.

Invoke · Collaborate

General

SUBAGENT

General research + multi-step task delegation

Explore

SUBAGENT

Read-only exploration — ideal for navigating large codebases

OpenCode / Token

Direct passthrough: transparent, flexible, optimizable

Token flow

1 **OpenCode CLI**
Orchestration only

2 **Provider API**
Claude / GPT / Local

3 **Your wallet**
Pay-per-Token

Fully transparent

No platform markup — model vendor pricing passthrough

Controllable levers

Pick model / cache / sub-agents to amortize cost

Data autonomy

Route via your own gateway — unified quota and auditing

Note: Multi-turn Agent loops easily amplify Tokens with large context — pair with Plan mode and local caching.

Chapter 3 / OpenClaw

The beauty of minimalism: AI on WhatsApp, powered by Pi

OpenClaw

WhatsApp · Personal AI assistant

Launched by Peter Steinberger, viral over the holidays; underneath is Mario Zechner's minimalist Coding Agent — Pi.

01 **Only 4 tools**

read · write · edit · bash — kernel minimalism taken to the limit

02 **Self-rewriting**

Need new features? Have Pi modify Pi itself — non-engineers can extend it

03 **AI restraint**

Refuses to pile every fancy capability into a single binary

OpenClaw / Philosophy

Why is "less is more"?

> *AI tools should be restrained, not endlessly inflated. We give the most complex part to the model, and the simplest part to the user.*
>
> — Mario Zechner, author of Pi

Traditional Agent	OpenClaw / Pi
Dozens of built-in tools, steep learning curve	**4 tools · 5 minutes to onboard**
New need = wait for the developer to ship a version	**New need = have Pi rewrite itself**
Complex UI, complex menus	**One WhatsApp message**

OpenClaw / Token

Minimal kernel = minimal Token baseline

4

built-in tools

Fewer tool schemas injected into the prompt → system-prompt tokens noticeably lower than peers

Token profile

+ **System prompt is extremely light**
Few tool descriptions — context window stays free for business

+ **BYOM — you pay**
Token cost is entirely on the user's LLM account

+ **Auditable, rewritable**
Open source + self-rewrite — easy to see exactly which part burns tokens

– **Complex tasks need multiple rounds**
Fine-grained tools mean complex tasks may loop multiple times

Chapter 4 / GitHub Copilot

From code completion to multi-form Coding Agent

2021	2024	2025	2026
Code completion	**Chat & Spaces**	**Agent Mode**	**Token era**
Copilot launches with editor ghost text	Conversational assistance, multi-model choice	In-IDE multi-step agent + Cloud Agent	Quotas shift from Request to Token

Two Agent forms coexist

· IDE Agent Mode: in-editor interactive multi-step agent

· Copilot Cloud Agent: runs in the background on GitHub Actions, handling Issues and PRs

Copilot / Agent forms

IDE interactive vs. Cloud background delegation

Agent Mode (IDE)

INTERACTIVE

- Runs in VS Code / Visual Studio / JetBrains
- Can read/write files, run terminal, call MCP tools
- Each user-initiated request = 1 Premium Request
- Suited to real-time iteration and visual verification

Cloud Agent (remote)

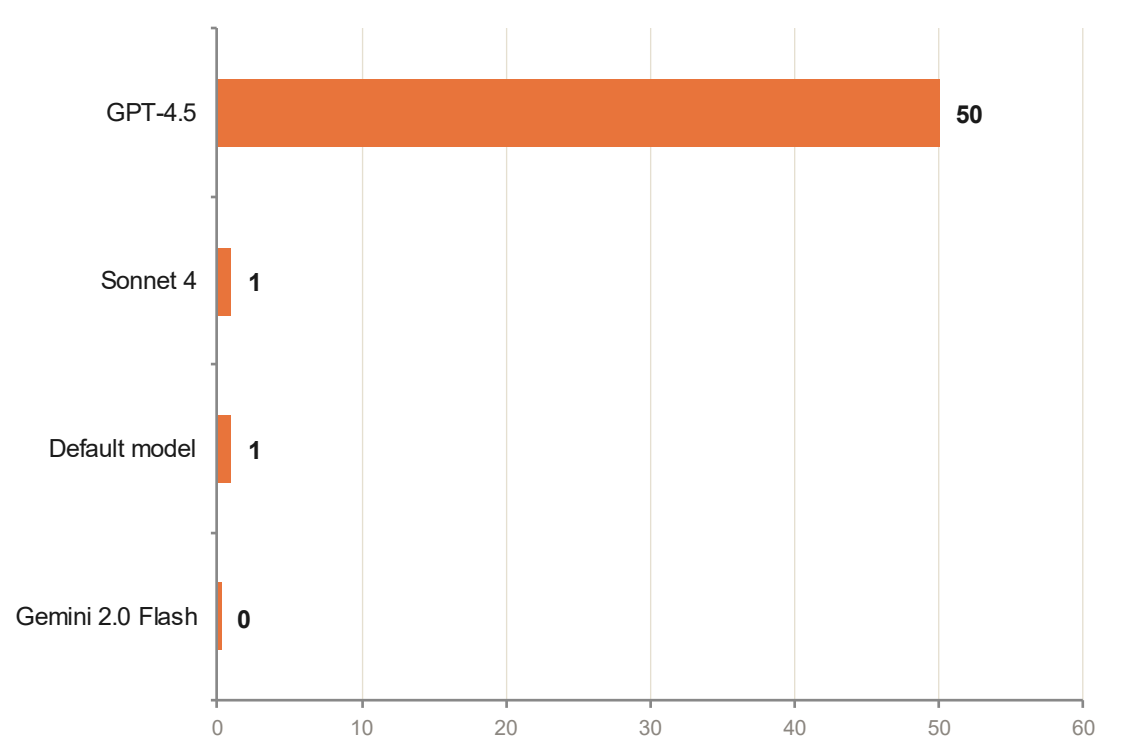
DELEGATED

- Delegated to an Issue / PR, executed in the background via Actions
- Whole Session = 1 Premium Request (fixed)
- Consumes GitHub Actions minutes during the Session
- Suited to long tasks, batch changes, DRI async collaboration

Copilot / Billing

Premium Request and model multipliers

Model multipliers (examples)



Monthly Premium quota by plan

Plan	Monthly quota
Free	50
Pro	300
Pro+	1,500
Business	300 / user
Enterprise	1,000 / user

Overage: $0.04 / Premium Request — one 50× model call = $2.00

Starting June 2026, Copilot will gradually shift to Token-based quota metering.

Side-by-side comparison

Three Token billing philosophies

Dimension	OpenCode	OpenClaw / Pi	GitHub Copilot
Billing model	Direct model API passthrough	Direct model API passthrough (BYOM)	Premium Request → Token
Transparency	High · every token visible	High · small kernel, easy to audit	Medium · multipliers and quotas need manual checking
Platform markup	None	None	Yes (subscription + overage)
Optimization levers	Pick model / cache / sub-agents	Rewrite Pi itself, custom tools	Pick model, cap Agent steps
Typical cost structure	Per-token, two lines (input + output)	Per-token, two lines, low baseline	Fixed plan + $0.04 overage
Large-context impact	Obvious — shows up directly on the bill	Obvious, but tool context is small	Reflected via Token quota

Token Consumption Profile

Break the bill apart

OpenCode

High freedom · high responsibility

100%

Tokens come directly from the vendor bill

- Context grows with the project — must be actively trimmed
- Plan mode is read-only — reduces accidental-action tokens
- Use an AI gateway for caching and unified billing

OpenClaw / Pi

Minimal kernel · low baseline

4

tools, minimal tool context

- Very few tool schemas — lowest system-prompt overhead
- Handle complex tasks via self-rewrite, not more tools
- Cost structure follows whichever model the user picks

GitHub Copilot

Plan-wrapped · multiplier-amplified

1 → 50×

Model multipliers float dynamically

- Cloud Agent counts whole Session as 1 — strong price-performance
- Premium models gated by multipliers (up to 50×)
- Token quotas (2026) make long-context cost visible

Selection matrix

When to use which?

Scenario	Recommended	Reason
I want highly controllable LLM collaboration in the terminal	**OpenCode**	*Open source + BYOM, Plan mode auditable*
I want a lightweight, self-evolving personal Agent	**OpenClaw / Pi**	*Minimal kernel + self-rewrite*
I need to delegate background tasks on PRs / Issues	**Copilot Cloud**	*Session-based billing, deep GitHub integration*
I need team-level licensing, compliance, unified billing	**Copilot Enterprise**	*1,000 Premium / user, IT-friendly*
I want to run the Agent offline with local models	**OpenCode + Local LLM**	*Model-agnostic, supports Ollama and others*

Cost optimization

Spend every Token where it matters

01

Tier-pick the model

Frequent tasks on cheap models — pull out Opus / GPT-5 only at critical moments

02

Use the cache well

Anthropic cache-hit price is 1/10 of base — long system prompts benefit most

03

Plan first

Use OpenCode Plan / task decomposition first — then switch to Build

04

Gateway & quotas

Put an AI gateway in front of OpenCode / OpenClaw — centralized quotas and auditing

05

Delegate to Cloud Agent

Copilot Cloud counts the whole Session as 1 Premium — best value for long tasks

06

Cap Agent steps

Every step costs tokens — set a step cap + termination conditions for the Agent

Future trends

Token economics will reshape the developer market

01

Request billing → Token billing

Platforms like GitHub are gradually replacing fixed Requests with Token quotas — billing precision closer to actual compute.

02

Local-model resurgence

BYOM tools like OpenCode and Pi bring local inference back as an option — balancing data compliance and cost.

03

Agent of Agents

Build calls Plan, Plan calls Explore — sub-agent reuse splits one task into many small charges.

04

Minimalist-philosophy revival

OpenClaw / Pi's popularity shows: "four tools + one strong model" may be more sustainable than piling on tools.

Three-sentence wrap-up

If you only remember three things

01

Understand the Token flow

Whatever platform you use, first ask: "Who pays for this token, and who can see it?" OpenCode and Pi pass through directly; Copilot is wrapped in plans.

02

Embrace the minimal kernel

OpenClaw proves 4 tools are enough. Solve problems with the minimum set first, extend only when necessary — usually cheaper and more stable.

03

Design Agents for cost

Cap loop steps, introduce Plan mode, tier the model, cache system prompts — optimize Tokens at the engineering layer.

Thank You

"The best Agent isn't the one with the most tools — it's the one that spends Tokens most precisely."

QUESTIONS

Live Q&A / continued at the break

DEMO

Live comparison demo of all three platforms

CONTACT

Kinfey Lo · Microsoft Cloud Advocate